

ATI Jaffna Integrated Web Portal

- **Final Project Viva Presentation**
- A centralized academic, administrative, financial, content, and communication platform
- Technology: React + Vite | Node.js + Express | MongoDB
- Presented by: _____ Registration No: _____

Presentation Roadmap

- Problem and motivation
- Objectives, scope, and users
- System architecture and technology choices
- Major functional modules and workflows
- Database, API, security, testing, and deployment
- Challenges, limitations, future work, and viva questions

Institutional Problem

- Public information and internal academic services are often separated or handled manually.
- Students need one place for courses, attendance, grades, assignments, fees, notices, and support.
- Lecturers need department-scoped tools for students, learning materials, assessment, and communication.
- Administrators need centralized control of users, content, reports, donations, settings, and system status.
- Manual and fragmented processes reduce visibility, consistency, speed, and accountability.

Proposed Solution

- A role-based full-stack web application for ATI Jaffna.
- Combines a public institutional website with authenticated portals.
- Digitizes academic workflows, fee administration, CMS publishing, donations, and messaging.
- Provides an AI assistant using institutional and live portal knowledge.
- Uses a modular REST API and MongoDB persistence for extensibility.

Project Objectives

- Create a responsive, accessible institutional website.
- Provide secure role-specific dashboards for students, lecturers, department staff, finance staff, and administrators.
- Reduce repetitive manual work through CRUD, bulk import, reports, receipts, and automated status calculations.
- Protect departmental and personal data through scoped authorization.
- Support maintainability with modular code, validation, logging, health checks, and automated tests.

Users and Access Roles

- Public visitor: views institutional content, courses, staff, news, notices, campaigns; submits contact messages and donations.
- Student: accesses own academic, LMS, attendance, grades, assignment, fee, and communication data.
- Lecturer / department staff: manages only permitted department students, courses, materials, assessment, and communication.
- Finance officer: manages institution-wide fees, payments, refunds, reporting, and analytics.
- Administrator: full management of users, academic records, CMS, donations, settings, analytics, and maintenance mode.

Project Scope

- Public website: Home, About, Faculties, Courses, News, Contact, Donation Wall, and donation flow.
- Academic portal: students, faculty, courses, timetable, attendance, grades, assignments, forms, and study materials.
- Administration: user approval, CRUD management, CMS drafts/publishing, analytics, settings, and maintenance mode.
- Finance: fee structures, invoices, payments, receipts, refunds, outstanding balances, service requests, and audit logs.
- Intelligence: public and authenticated AI chat with uploaded institutional knowledge.

Technology Stack

- Frontend: React 19, React Router, Vite, Tailwind CSS, Material UI, Lucide icons, Framer Motion.
- Data/UI: Axios, TanStack Query, Recharts, QRCode, jsPDF.
- Backend: Node.js, Express 5, Mongoose, JSON Web Tokens, bcrypt, Multer.
- Database: MongoDB with schemas, indexes, references, and timestamps.
- AI/document processing: Groq/NVIDIA-compatible chat APIs, PDF parser, Mammoth DOCX parser, PPTX ZIP/XML extraction.
- Quality/security: Node test runner, Helmet, CORS, rate limiting, sanitization, logging, request IDs.

High-Level Architecture

- Client layer: responsive React single-page application with public and protected routes.
- State layer: authentication, theme, language, modal contexts, and server-state query caching.
- API layer: Express REST routes grouped by business module.
- Business layer: controllers and services enforce validation, workflows, and role/department scope.
- Data layer: Mongoose models persist users, academics, finance, CMS, donations, AI knowledge, and audit records.
- External layer: AI provider APIs and browser geolocation for attendance.

Repository and Implementation Scale

- Separate Frontend and Backend applications managed from the project root.
- 38 backend model files, 18 route modules, and 15 controller modules.
- 35 frontend page components plus shared layouts, contexts, hooks, and reusable components.
- Approximately 143 explicit API route declarations identified in the source.
- 129 principal source files across backend modules and frontend pages/components.
- Modular organization makes features easier to test, maintain, and extend.

Frontend Design

- React Router separates public, student, faculty, finance, and admin route trees.
- Lazy loading and Suspense reduce initial page loading cost.
- DashboardLayout provides role-aware navigation; MainLayout serves public pages.
- AuthContext holds session state; protected routes redirect unauthorized users.
- Theme and language contexts support user preferences and localization structure.
- TanStack Query and API helper modules organize asynchronous server data.

Backend Design

- Express server applies security, parsing, tracing, logging, rate limiting, static uploads, routes, and centralized errors in order.
- Routes define HTTP contracts; controllers implement domain behavior; services hold reusable business logic.
- Generic resource routes reduce duplicated CRUD code for suitable collections.
- Dedicated controllers handle complex modules such as authentication, fees, assignments, grades, attendance, contacts, and AI.
- Health, readiness, and operational metrics endpoints support deployment monitoring.
- Graceful shutdown closes resources safely.

Database Design

- Identity: User, Student, Faculty, Admin, DepartmentStaff, and AdminStaff.
- Academics: Department, Course, Semester, TimetableEntry, AttendanceRecord, GradeRecord, Assignment, Form, and StudyMaterial.
- Finance: StudentFee, FeeCategory, FeeStructure, Invoice, Payment, Receipt, Refund, Notification, FeeServiceRequest, and AuditLog.
- Content/community: PageContent, PortalData, Blog, Notice, Event, Contact, Donation, and DonationCampaign.
- AI: KnowledgeDocument and KnowledgeChunk.
- Indexes support identity uniqueness, role/status filtering, and department-based queries.

Authentication Workflow

- Student registration validates mandatory identity and academic fields.
- A Student profile and User credential record are created together; rollback removes partial records on failure.
- New student accounts remain pending until administrator approval.
- Login accepts email or Student ID, verifies bcrypt hash and account status, then issues a signed JWT.
- JWT is sent as a Bearer token for protected API calls.
- Forced password change and profile-photo update workflows are supported.

Authorization and Data Isolation

- Frontend route guards improve navigation and user experience.
- Backend `requireAuth` verifies JWT; `requireRole` and `requireAdmin` enforce server-side privileges.
- Department middleware limits lecturers and department staff to their assigned department.
- Students are restricted to their own fee and academic records.
- Admin and finance roles can access institution-wide financial data.
- Notice and contact visibility is calculated by audience, role, sender, and department.

Student Portal

- Dashboard summarizes the student workspace.
- Course/LMS pages display enrolled or stage-relevant courses and downloadable study materials.
- Timetable, attendance, grades, and GPA-related information are available from protected APIs.
- Assignments support files, submission, comments, and review status.
- Fees show student-specific summaries, payments, invoices, receipts, refunds, and requests.
- Messages, announcements, profile management, and AI assistance complete the experience.

Faculty and Department Portal

- Faculty dashboard presents department-relevant information.
- Authorized staff can view students only within their department.
- Timetable tools use permitted lecturer/course options.
- Attendance, grades, and assignments support teaching workflows.
- Faculty course pages upload, update, list, download, and delete study materials.
- Department-scoped AI knowledge and messages reduce cross-department data exposure.

Admin Portal

- Dashboard and analytics provide institutional oversight.
- User management supports account creation, approval, editing, and deletion.
- Student, faculty, course, timetable, grade, and fee management centralize academic administration.
- CMS supports draft saving, publishing, unpublishing, and controlled image uploads.
- Donation administration includes dashboard, donor list, reports, campaigns, and settings.
- System settings include institution data, integrations, reset operations, and maintenance mode.

Part-Time Fee Management

- Defines categories and fee structures, then assigns student fee obligations.
- Generates invoices and records one or multiple payments.
- Payment-status service calculates pending, partial, paid, or not-required states.
- Produces downloadable receipts and public receipt verification codes.
- Handles outstanding balances, refunds, notifications, and service requests.
- Reports and audit logs improve accountability for finance operations.

Attendance, Grades, and Assignments

- GPS attendance compares browser coordinates with configured campus latitude, longitude, and allowed radius.
- Attendance session rules and protected record listing reduce invalid marking.
- Grades support individual CRUD and bulk import with department/student checks.
- Assignments support creation, attachments, comments, submissions, lecturer review, updates, and deletion.
- Forms can be linked to assignments and collect structured responses.
- These modules replace disconnected spreadsheets and paper-based tracking.

CMS, Notices, Contact, and Donations

- CMS separates draft content from published public pages.
- Notices are filtered for all visitors, students, lecturers, or administrators.
- Contact/complaint records are routed to administration or the appropriate department.
- Donation campaigns and the public donor wall encourage community participation.
- Donation sessions, verification, receipts, reports, and campaign administration form an end-to-end flow.
- Maintenance mode temporarily protects public access while allowing admin entry.

AI Assistant and Knowledge Retrieval

- Accepts PDF, DOCX, PPTX, and TXT institutional documents.
- Extracted text is normalized, split into overlapping chunks, and stored as knowledge records.
- A lightweight term-frequency vector and cosine similarity rank relevant chunks.
- The prompt combines retrieved knowledge with authorized live portal data.
- Scope depends on requester role and department; public chat uses public-safe context.
- Configured Groq or NVIDIA-compatible models generate the final response.

Security Controls

- Passwords are hashed with bcrypt; tokens are signed with a configurable JWT secret.
- Helmet and custom headers reduce browser-based attack exposure.
- CORS allow-list controls browser origins; JSON body limits and rate limiters reduce abuse.
- Request sanitization removes MongoDB operator and dotted keys to mitigate NoSQL injection.
- Uploads validate extension and MIME type, limit size, and sanitize filenames.
- Central error handling avoids inconsistent failures; request IDs and logs improve traceability.

API and Operational Reliability

- REST resources use clear GET, POST, PUT/PATCH, and DELETE operations.
- Central asynchronous error forwarding and standardized 404 responses simplify clients.
- Health endpoint reports API status, database state, ping time, uptime, environment, version, and memory.
- Readiness endpoint prevents traffic before MongoDB is connected.
- The development server can auto-start a local MongoDB instance when configured.
- Graceful shutdown and structured logging support safer operations and diagnosis.

Testing and Verification

- Executed the complete backend suite using: `npm run test:all`.
- Result: 29 tests passed, 0 failed, 0 skipped.
- Verified department access, fee access, contact visibility, and notice audiences.
- Verified fee payment/refund status calculations and full-time exemptions.
- Verified Mongo request sanitization, settings validation/default merging, and safe upload names.
- Limitation: current automated suite focuses on backend units; browser E2E, load, and full integration tests remain future work.

Key Design Decisions

- MERN-style stack provides one primary language across client and server.
- Role plus department scope matches the institution's organizational structure.
- JWT enables stateless API authentication and simple SPA integration.
- MongoDB supports varied academic, content, financial, and AI document shapes.
- Lazy-loaded portals keep a large frontend manageable.
- Custom retrieval avoids requiring a separate vector database, but is less semantic than embeddings.

Challenges and Solutions

- Challenge: preventing cross-department data leakage. Solution: shared department and fee-access middleware plus tests.
- Challenge: synchronizing User, Student, and Faculty identities. Solution: profile services, sync scripts, uniqueness indexes, and rollback logic.
- Challenge: complex fee state changes. Solution: centralized recalculation service and refund/payment tests.
- Challenge: different office document formats. Solution: format-specific extraction and normalized chunk indexing.
- Challenge: running locally. Solution: root start command, environment-based configuration, and optional MongoDB auto-start.

Current Limitations

- No production payment-gateway integration is evident; donation verification uses application-managed flows.
- Term-frequency retrieval can miss semantic matches and is less scalable than embedding/vector search.
- JWT storage/revocation and refresh-token strategy should be strengthened for production.
- Some broad modules need more controller integration and browser end-to-end tests.
- Local upload storage requires object storage, malware scanning, and lifecycle policies at production scale.
- Deployment automation, backups, disaster recovery, and formal performance benchmarks are not yet demonstrated.

Future Enhancements

- Add email/SMS/push notifications and calendar synchronization.
- Integrate a certified online payment gateway and reconciliation webhooks.
- Use embeddings plus a vector database for multilingual semantic retrieval.
- Introduce refresh tokens, optional MFA, fine-grained permissions, and session revocation.
- Add Playwright/Cypress E2E tests, API integration tests, load tests, and CI/CD quality gates.
- Deploy using containers, managed MongoDB, object storage, monitoring, backups, and HTTPS.

Project Outcomes

- One platform now represents public information and core institutional workflows.
- Five role categories receive purpose-specific access and navigation.
- Academic, finance, CMS, donation, and AI features share a consistent API and data layer.
- Security controls are applied globally and at business-module boundaries.
- The verified backend suite passes all 29 available tests.
- The architecture provides a practical foundation for production hardening and institutional expansion.

Conclusion

- The project solves a genuine institutional coordination problem with an integrated portal.
- Its strongest qualities are breadth, modularity, role/department isolation, and operational awareness.
- The solution demonstrates full-stack engineering: UI, API, database, security, testing, and AI retrieval.
- The next step is production hardening through stronger identity, payments, storage, testing, and deployment infrastructure.
- Thank you — Questions and demonstration.

Likely Viva Questions: Architecture

- Why React? Component reuse, SPA navigation, ecosystem support, and role-specific route composition.
- Why MongoDB? Flexible schemas for varied portal domains and rapid iteration; indexes preserve important query performance and uniqueness.
- Why REST? Simple resource-oriented contracts, broad client/tool compatibility, and easy separation of frontend and backend.
- Why separate controllers and routes? Routes define endpoints; controllers hold business behavior, improving maintainability and testing.
- Why JWT? Stateless authentication fits a separate SPA/API, though production should add refresh/revocation controls.

Likely Viva Questions: Security and Quality

- Can frontend guards secure data? No. They only improve UX; backend middleware is the real security boundary.
- How is NoSQL injection reduced? Operator/dotted-key sanitization, controlled query construction, schema validation, and authorization.
- How are uploads protected? Extension plus MIME allow-lists, size limits, safe generated filenames, and restricted static serving.
- How was correctness verified? Source review, modular tests, and a full run: 29 passed, 0 failed.
- What would you test next? Authentication integration, complete workflows, browser E2E, concurrency, load, accessibility, and recovery.

Demonstration Sequence

- 1. Open public Home, Courses, Faculties, News, and Contact pages.
- 2. Register/login as student and show dashboard, course materials, attendance, grades, assignments, and fees.
- 3. Login as faculty and demonstrate department-scoped students and learning-material management.
- 4. Login as finance officer and show invoice, payment, receipt, refund, and report flows.
- 5. Login as admin and show approvals, CMS publishing, analytics, donations, settings, and maintenance mode.
- 6. Upload a knowledge document and ask the AI assistant a scoped question.